

Module 1: Introduction to Mobile Development and Flutter

1. Explain the benefits of using Flutter over other cross-platform frameworks

Flutter, developed by Google, offers several benefits over other cross-platform frameworks like React Native, Xamarin, or Ionic:

- **Single Codebase for Multiple Platforms:** Flutter allows developers to write one codebase for Android, iOS, web, desktop, and embedded platforms, reducing development time and cost.
- **High Performance:** Flutter uses its own rendering engine (Skia), allowing it to compile to native ARM code. This results in near-native performance, unlike frameworks that rely on web views or JavaScript bridges.
- **Customizable UI with Widgets:** Flutter provides a rich set of customizable widgets that help developers create expressive and beautiful UIs that look consistent across platforms.
- **Hot Reload:** Developers can instantly see changes in the code without restarting the app, making it much faster to iterate and debug.
- **Strong Community and Ecosystem:** Google's backing and a growing developer community contribute to a rich package ecosystem and regular updates.
- **Access to Native Features:** Flutter provides platform channels to call native APIs, which makes it easy to access features like the camera, location, or sensors.

2. Describe the role of Dart in Flutter. What are its advantages for mobile development?

Dart is the primary programming language used in Flutter. It plays a crucial role in both the frontend (UI) and business logic of Flutter apps.

Advantages of Dart for Mobile Development:

Module 1: Introduction to Mobile Development and Flutter

- **Ahead-of-Time (AOT) Compilation:** Dart can compile to native code for fast startup and performance on mobile devices.
 - **Just-in-Time (JIT) Compilation:** Useful during development, JIT enables hot reload, making the dev process much quicker.
 - **Object-Oriented Language:** Dart's syntax is familiar to developers coming from Java, C#, or JavaScript, easing the learning curve.
 - **Built-in Asynchronous Support:** With `async/await`, Dart makes it easy to write clean asynchronous code for tasks like network calls.
 - **Unified Language:** Using Dart for both UI and business logic allows for better consistency and easier maintenance compared to frameworks that split UI and logic across different languages.
-

3. Outline the steps to set up a Flutter development environment

Here's how to set up Flutter on your system:

Step-by-Step Setup:

1. **Download Flutter SDK**
 - Go to <https://flutter.dev> and download the SDK for your operating system.
2. **Extract the SDK**
 - Extract the downloaded file to a suitable location (e.g., `C:\flutter` on Windows or `~/flutter` on macOS/Linux).
3. **Update System Path**
 - Add the Flutter `bin` directory to your system `PATH`.

Module 1: Introduction to Mobile Development and Flutter

4. Run Flutter Doctor

Open a terminal or command prompt and run:

```
flutter doctor
```

-
- This checks your environment and shows what's missing (e.g., Android Studio, Dart SDK, emulators).

5. Install Android Studio (or another IDE)

- Install Android Studio and its required plugins: Flutter and Dart.

6. Set Up an Emulator

- In Android Studio, create and start an Android emulator.

7. Run a Test App

Create a sample app:

```
flutter create my_app  
cd my_app  
flutter run
```

-
-

4. Describe the basic Flutter architecture

Flutter uses a layered architecture, which makes it easy to customize and extend. Here's a basic breakdown:

Basic Layers:

Module 1: Introduction to Mobile Development and Flutter

- **Framework Layer (Dart)**

- This is where you write your app, using Flutter's UI toolkit (widgets, gestures, rendering, etc.).

- **Engine Layer (C++)**

- The Flutter engine handles rendering (Skia), text layout, plugin architecture, and Dart runtime.

- **Embedder Layer**

- This bridges the Flutter engine with the target platform (Android, iOS, Windows, etc.). It uses platform-specific code (Kotlin, Swift, Java) to access native APIs.

Widget System:

Flutter apps are built using widgets. There are two main types:

- **StatelessWidget**: Immutable widgets that don't change after they're built.
- **StatefulWidget**: Widgets that maintain state and update when the state changes.

Widgets build a **widget tree**, which is rendered using a corresponding **element tree** and **render tree**.